

Authority-Preserving Edge Revocation for Federated AI-Agent Workflows

Anonymous

Anonymous Institution

Abstract—Modern agent frameworks compose planners, tool agents, remote services, and shared specialists into runtime delegation graphs. Agent-identity and protocol proposals help authenticate who an agent is and what scoped authority it carries, but they leave open a workflow question: after withdrawing one delegation, which agents should lose authority while independently authorized agents keep working? We study this *authority consistency* problem for federated AI-agent workflows modeled as scoped signed delegation DAGs. Under disjunctive authority, the exact target of revoking edge e is $T_{\text{intent}}(e, G) = \text{reach}(G) \setminus \text{reach}(G \setminus \{e\})$, the agents whose every authorizing root path used e . We promote this standard reachability fact into a verifier-checkable authority-withdrawal contract for signed agent delegation: tree cascades over-revoke shared agents, while deployer-scoped cascades under-revoke cross-domain descendants. In a LangGraph framework-replay study, 25 workflow/input cells repeated 20 times yield 500 compiled-framework traces, 10,500 raw runtime events, and 2,000 valid signed delegation decisions; 13/25 cells contain runtime multi-parent sharing and 8/25 contain cross-deployer sharing. Signed replay verifies 500/500 target proofs, preserves all 320 alternate-parent shared-agent cases that tree cascade revokes, and rejects unauthorized signers and omission attacks. Baseline replay over 1,960 edge withdrawals shows that holder/node and tree-style targets cannot express the required authority-preserving behavior; small A2A/AutoGen/CrewAI portability artifacts, including a pinned executable CrewAI workflow, emit 25/25 valid signed events through the same schema.

Index Terms—AI agents, multi-agent systems, trustworthy AI, delegation, authorization, revocation, agent workflows

I. INTRODUCTION

AI-agent workflows are becoming multi-principal systems rather than single assistant sessions. Frameworks such as AutoGen [1], CrewAI [2], and LangGraph [3] compose planners, tool agents, remote specialists, and shared service agents; interoperability protocols such as A2A [4] and MCP [5] make those handoffs cross process and organizational boundaries. Recent agent-identity work argues that agents need authenticated, authorized, and auditable delegated authority [6]–[8]. This paper asks the next workflow question: if one delegation relationship is withdrawn, which agents should lose authority and which should continue through independent delegation paths?

The ambiguity is semantic. Consider a node C with two delegations $A \rightarrow C$ and $B \rightarrow C$: if A revokes only its edge, a disjunctive policy keeps C authorized through $B \rightarrow C$, so a tree cascade from C over-revokes C and its descendants. Conversely, if $A \rightarrow C \rightarrow D$ crosses into another deployer domain, a cascade restricted to A 's registry may revoke C but

miss D (under-revocation). We therefore argue that revocation must specify its semantic target before choosing a propagation mechanism. Under disjunctive authority—a node is authorized iff some valid root-to-node path survives—revoking edge e should invalidate exactly the nodes that lose all root paths when e is removed,

$$T_{\text{intent}}(e, G) = \text{reach}(G) \setminus \text{reach}(G \setminus \{e\}),$$

the nodes edge-dominated by e . The graph characterization is standard; the contribution is to promote it into the target language that signed agent protocols must carry, prove what evidence lets a relying party check it, and measure when common target languages are unsafe or disruptive. Figure 1 shows the failure on concrete multi-agent handoff graphs: a shared specialist should survive if another workflow still delegates to it, while a remote descendant should be revoked if its only authority runs through the withdrawn edge.

a) Contributions.

- **Authority-consistency contract.** A scoped signed-DAG model for federated AI-agent delegation, where the edge-revocation target is the edge-dominated set $\text{reach}(G) \setminus \text{reach}(G \setminus \{e\})$; the novelty is not a new dominator algorithm but a verifier-checkable revocation contract for signed agent authority.
- **Agent evidence and baselines.** Executed LangGraph traces are normalized into signed delegation decisions, then replayed against exact, holder/node, tree, deployer-scoped, and full-graph revalidation target languages; small A2A-, AutoGen-, and CrewAI-shaped artifacts plus a pinned executable CrewAI workflow emit the same normalized schema.
- **Proof-carrying evidence.** Ed25519-signed edges, a history-binding revocation log, and a relying party that checks the batch target $\text{reach}(G) \setminus \text{reach}(G \setminus R)$ from signed survivor/cut evidence, without storing the full graph.

Propagation/delivery is an orthogonal layer and out of scope here; we fix the target semantics it must carry.

b) Scope. We do not design a Byzantine propagation protocol, a transparency log, or a production agent platform. We also do not claim that disjunctive authority is the only possible agent policy. Conjunctive, threshold, and path-scoped authority policies are valid alternatives, but they induce different target functions. The contribution here is to make one common

TABLE I

ADJACENT MECHANISMS AUTHENTICATE CREDENTIALS, AGENT IDENTITY, OR COMMUNICATION ENDPOINTS, BUT DO NOT DEFINE THE EXACT AUTHORITY LOSS CAUSED BY ONE REVOKED EDGE IN A MULTI-PARENT AGENT DAG.

Mechanism	Main target	Multi-parent	Exact edge target
CRL / OCSP [9], [10]	certificate	no	no
OAuth / MCP [5], [15]	token/scope	no	no
Agent IAM [6], [7]	agent identity	partial	no
AIP [8]	delegation token	chain	no
A2A [4]	agent endpoint	yes	no
APS [16]	subtree (tree)	no	no
This paper	delegation edge	yes	yes

policy contract explicit and measure what happens when agent systems approximate it.

II. BACKGROUND AND MOTIVATION

A. From Chains to Multi-Parent DAGs

Many agent-identity and delegated-authorization proposals naturally emit chains: a user delegates to an agent, that agent invokes a tool or another agent, and a verifier checks the sequence of signed or token-bound authority statements. This is enough when each task has one parent at a time.

The graph model captures authorization patterns that linear chains cannot represent. A task-execution agent may be authorized by both a scheduling service and a billing service. A shared infrastructure agent may be admitted by multiple orchestrators. A cross-deployer task may pass from a root agent operated by one organization to a worker operated by another. We model these relationships as a directed acyclic graph rather than assuming that each authorization proof is a single chain. The multi-parent property, $|\text{parents}(v)| > 1$, is the case in which revocation semantics become nontrivial.

B. Why Existing Revocation Mechanisms Are Not Enough

Classical PKI mechanisms (CRLs [9], OCSP [10], CR-Lite [11]) are credential-centric: a certificate binds one subject to one key and revocation invalidates that binding, with no language for revoking one delegation relationship while preserving another. Capability systems such as Macaroons [12] support attenuation but revoke via short-lived tokens or root re-contact; workload-identity systems such as SPIFFE [13] rely on short lifetimes; and formal models such as DW-RBAC [14] assume a centralized engine in a single domain—whereas agent delegation needs relying parties to verify authority and revocation from signed evidence without a central registry at decision time. Table I makes the gap concrete: PKI and capabilities define credential or token targets; MCP and A2A define communication and tool/agent access surfaces; recent agent-identity systems authenticate delegated authority; and APS-like tree passports support cascade on rooted structures. None gives an authority-preserving target for revoking one edge in a multi-parent, cross-deployer agent DAG.

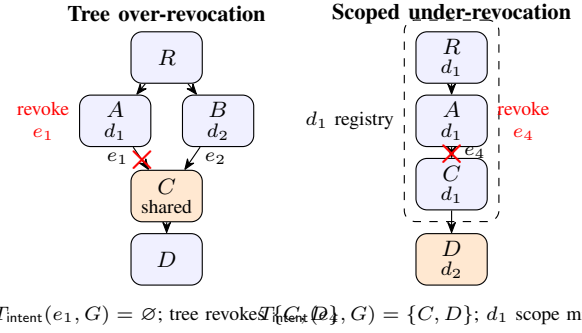


Fig. 1. Two deterministic failure witnesses. Left: revoking $A \rightarrow C$ in a shared-agent DAG leaves C reachable through $B \rightarrow C$, so tree cascade over-revokes C and D . Right: revoking $A \rightarrow C$ in a cross-deployer chain disconnects C and D , but a cascade limited to the d_1 registry revokes only C and misses the cross-domain descendant D .

C. Two Failure Modes

Figure 1 illustrates the two core failures. *Over-revocation*: in root $\rightarrow A, B$; $A, B \rightarrow C$; $C \rightarrow D$, revoking $A \rightarrow C$ leaves C authorized through $B \rightarrow C$, yet a tree cascade from C revokes $\{C, D\}$, causing collateral disruption. *Under-revocation*: in a cross-deployer chain root(d_1) $\rightarrow A(d_1) \rightarrow C(d_1) \rightarrow D(d_2)$, revoking $A \rightarrow C$ should remove both C and D , but a d_1 -scoped cascade sees C and misses the cross-domain descendant D .

III. MODEL AND SEMANTICS

A. SIGNED-DAG Delegation Model

Definition 1 (SIGNED-DAG). A SIGNED-DAG is a tuple $G = (V, E, \sigma)$ where V is a finite set of agent nodes, $E \subseteq V \times V$ is a set of directed delegation edges, and σ maps each edge to a deployer signature over the edge descriptor. Each node v has a deployer label $\text{dep}(v)$ and public key $\text{pk}(v)$. An edge $e = (p, c)$ means that parent p delegates authority to child c . The graph is acyclic.

Roots are trust anchors known to verifiers out of band. We write $\text{Roots}(G)$ for the root set and $\text{reach}(G)$ for the nodes reachable from any root by following directed edges in G .

The signatures in σ authenticate the graph. They are not what makes the edge-dependence result true; the result is graph reachability. Signatures make the result enforceable without contacting a central authority, because a relying party can check the authenticity of edges and revocation events locally.

B. Revocation Log

Let $\mathcal{R}$ be a signed revocation log containing pairs (e, t_e) , where e is an edge identifier and t_e is the revocation time. The edge predicate is:

$$\text{REVOKED}(e, t) \Leftrightarrow \exists (e', t_e) \in \mathcal{R} : e' = e \wedge t_e \leq t.$$

Let $\mathcal{R}^t = \{e : \text{REVOKED}(e, t)\}$ be the set of edges revoked by time t .

TABLE II
THREAT ASSUMPTIONS AND CONTRACT BOUNDARY.

Condition	Treatment	Boundary
Forged delegation or revocation	reject by pinned deployer key	covered
Unauthorized revoker	parent domain must sign envelope	covered
Omitted in-edge to target node	signed in-edge commitment must match	covered
Log rollback or substitution	hash-bound monotone checkpoint detects mismatch	covered after observation
Delayed or suppressed update	propagation budgets δ_{prop}	out of scope
Byzantine deployer or privacy leak	not solved; disclosure is measured	out of scope

Definition 2 (Path validity). A root-to-node path $\pi = \langle e_0, \dots, e_k \rangle$ is valid at time t iff every edge is authenticated and unrevoked:

$$\text{valid}(\pi, \mathcal{R}, t) \Leftrightarrow \forall i \in [0, k]. \neg \text{REVOKED}(e_i, t) \wedge \text{sig_ok}(e_i).$$

C. Disjunctive Authority

Definition 3 (Disjunctive authority). A node v is active at time t iff at least one valid root-to- v path exists:

$$\text{active}(v, G, \mathcal{R}^t) \Leftrightarrow v \in \text{reach}(G \setminus \mathcal{R}^t).$$

This policy says that independent parent delegations are alternatives: a node retains authority if one valid parent path survives. The policy matches settings where multiple orchestrators can independently authorize a shared agent. It does not cover threshold approval or all-parent approval; those policies require different target functions.

D. Threat Model and Trust Assumptions

The contract is verifier-independent only relative to an authenticated prefix of the signed delegation graph and revocation log. A relying party pins deployer public keys out of band, verifies each edge and revocation envelope locally, and then checks the target without querying a central authorization service. The same paragraph-level assumptions are summarized in Table II.

Thus Theorem 1 is a correctness statement over the *observed, authenticated* graph: $\text{reach}(G) \setminus \text{reach}(G \setminus R)$ is exact for the graph and revocation set the verifier has seen. The prototype (Section VII) demonstrates signature checks, rollback detection, and authenticated adjacency completeness. It does not solve propagation freshness, transparency-log anchoring, Byzantine deployers who sign conflicting evidence, or confidentiality beyond the disclosure measurements.

IV. REVOCATION TARGETS

A. Design Requirements

Before defining the target we state the requirements a revocation mechanism for complex multi-parent agent systems must satisfy; the rest of the paper is organized around discharging them.

R1 (Edge-local intent). Name the specific delegation edge withdrawn—not a whole credential, principal, or subtree—so that authority granted through other edges is unaffected.

R2 (Multi-parent preservation). Alternate authorizing root path after a revocation must stay active; the mechanism must not over-revoke shared agents.

R3 (Cross-domain consistency). Authorizing paths must be revoked even when that status depends on edges crossing deployer/registry boundaries; the mechanism must not under-revoke across domains.

R4 (Verifier independence). A relying party must compute and check the target from signed evidence it already holds, with no online query to a central authority at decision time.

R5 (Batch revocation). Revocation of several edges must yield the exact joint target, which is not in general the union of the single-edge targets.

The disjunctive-authority target defined next is the policy that meets R1–R3; the verifier-independent prototype (Section VII) discharges R4; and batch non-compositionality (Section IV-E) addresses R5.

B. Semantic Target for Edge Revocation

Definition 4 (Edge-removal intent). For a single edge $e \in E$, the intended target set under disjunctive authority is:

$$T_{\text{intent}}(e, G) = \text{reach}(G) \setminus \text{reach}(G \setminus \{e\}).$$

This is the set of nodes that had authority before e was removed and do not have authority after e is removed. It is not the subtree below the child of e , because descendants may retain authority through alternate paths.

Theorem 1 (Edge-dependence characterization). For every reachable node v and edge e ,

$$v \in T_{\text{intent}}(e, G) \iff \text{every root-to-}v \text{ path contains } e.$$

Proof. If $v \in T_{\text{intent}}(e, G)$, then v is reachable in G but unreachable after removing e . Therefore no root-to- v path avoiding e exists, so every root-to- v path contains e . Conversely, if every root-to- v path contains e , then removing e destroys all root-to- v paths. Since v was reachable before removal and is unreachable after removal, $v \in T_{\text{intent}}(e, G)$. $\square$

The target can be computed in $O(|V| + |E|)$ time by two forward reachability traversals: one on G and one on $G \setminus \{e\}$. For many queries, standard dominator data structures can precompute equivalent edge-dominance information. The graph characterization is standard; the contribution is to make it the *target language* that signed agent protocols must carry. Edge identifiers, signed revocation envelopes, in-edge completeness commitments, and proof-carrying target sets let a verifier check this target without an online central authorization service.

The contract is meaningful only if a relying party can realize it locally. The following proposition states the correctness guarantee that the threat model (Section III-D) and prototype (Section VII) are built to provide; it formalizes requirement R4.

Proposition 1 (Local observed-prefix exactness). Let a relying party hold an authenticated view G_v of the delegation graph in which (i) every edge verifies against its parent deployer’s pinned key and (ii) every reachable node carries its deployer’s signed completeness commitment to its accepted in-edge set. Let R be the set of revocations the party has observed as an authenticated prefix of the signed revocation log. Then the

target computed locally, $\text{reach}(G_v) \setminus \text{reach}(G_v \setminus R)$, equals the intended target $T_{\text{intent}}(R, G)$ for that observed prefix—with no online query to any authority.

Proof. By edge authenticity, G_v contains no forged edge. By authenticated adjacency completeness, no genuine in-edge of a reachable node is omitted from G_v : a missing in-edge would contradict that node deployer’s signed commitment. Hence G_v and G induce the same root-reachability relation on reachable nodes, both on the full graph and after deleting R . The target is a function of those two reachability relations (Theorem 1), so the locally computed value equals $T_{\text{intent}}(R, G)$. Exactness is relative to the observed prefix R ; revocations not yet observed are the concern of the propagation layer and are bounded by that layer’s stale-authority budget. $\square$

C. Approximation Mechanisms

We compare mechanisms by the set of nodes they revoke for an edge-removal request.

a) *Edge target.*: The exact edge target computes $T_{\text{intent}}(e, G)$ and is the reference target for all baseline comparisons.

b) *Tree cascade.*: Tree cascade revokes the child of e and all descendants reachable by forward traversal. It is exact on trees, but over-revokes on DAGs whenever a descendant has a surviving alternate root path. This is not a synthetic strawman: it is the natural generalization of subtree revocation in tree-shaped delegation models, and the agent passport proposal (APS) [16] explicitly models delegation as a rooted tree with a single parent identifier, for which subtree cascade is the intended mechanism.

c) *Deployer-scoped cascade.*: Deployer-scoped cascade performs a forward traversal only through nodes visible to the initiating deployer. It can miss cross-domain descendants whose authority depends on the revoked edge. This models the common deployment reality that revocation is administered per administrative domain: deleting a Kubernetes `RoleBinding` affects only its namespace [17], OAuth token revocation is performed at the issuing authorization server [15], and SPIFFE/SPIRE trust domains issue and rotate credentials within a single domain [13]. None of these provides a cross-domain edge-revocation target.

d) *Node or deployer revocation.*: Node revocation invalidates all authority held by one node, regardless of which parent edge granted it—the model behind credential-centric revocation such as CRLs [9] and OCSP [10]. Deployer revocation invalidates an entire deployer domain. These mechanisms are appropriate for different operator intents, such as key compromise, but they are coarse approximations for removing one delegation relationship.

e) *Segment revocation.*: Segment revocation targets content-addressed output segments, not delegation authority. It is a useful primitive for output binding, but it is not evaluated as a substitute for edge revocation.

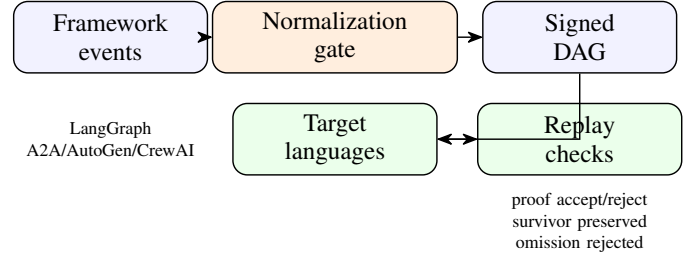


Fig. 2. Replay pipeline. E16 uses executed LangGraph traces as primary evidence; E17/E18 use protocol/framework-shaped handoffs and E19 uses a pinned CrewAI executable artifact to test whether the same signed-event schema is portable beyond LangGraph.

D. Error Metrics

Definition 5 (Over-revocation and under-revocation). For mechanism M and semantic target S :

$$\begin{aligned} \text{OverRev}(M, G, S) &= |M(G) \setminus S|, \\ \text{UnderRev}(M, G, S) &= |S \setminus M(G)|. \end{aligned}$$

Over-revocation is an availability and correctness failure: legitimate agents lose authority. Under-revocation is a security failure: agents that should lose authority remain active.

E. Batch Revocation Does Not Compose by Union

Revocation systems often process events independently. In multi-parent DAGs, that is not generally correct.

Proposition 2 (Batch target). For a set of revoked edges $R \subseteq E$:

$$T_{\text{intent}}(R, G) = \text{reach}(G) \setminus \text{reach}(G \setminus R).$$

In general,

$$T_{\text{intent}}(R, G) \neq \bigcup_{e \in R} T_{\text{intent}}(e, G).$$

Proof. Let root delegate independently to A and B , and let both A and B delegate to C . Let $R = \{A \rightarrow C, B \rightarrow C\}$. Removing either edge alone does not disconnect C , so both single-edge targets are empty. Removing both edges disconnects C , so $T_{\text{intent}}(R, G)$ contains C . Therefore the batch target is not the union of single-edge targets. $\square$

This result is operationally important. Even a system that computes exact single-edge targets can under-revoke if it processes simultaneous revocations by unioning independently computed single-edge targets.

V. MEASUREMENT METHODOLOGY

Figure 2 summarizes the measurement pipeline. Runtime framework events are not counted directly as delegations: they first pass a normalization gate requiring caller, callee, permission tuple, and deployer labels. The resulting signed delegation DAG is replayed against exact and approximate target languages.

A. Research Questions

We evaluate four questions aligned with the AI-agent framing.

- RQ1. When do cascade approximations deviate from the edge-removal semantic target?
- RQ2. How large is the deviation under multi-parent graph families?
- RQ3. How does batch revocation differ from the union of single-edge targets?
- RQ4. Do executed agent workflows exhibit the multi-parent and cross-deployer structures under which approximations fail, and can signed replay verify exact targets?

B. Graph Families

We combine several evidence sources. Hand-built witnesses isolate over- and under-revocation. Random and scale-free DAGs test whether failures persist beyond the witnesses. Batch-revocation tests sample multi-edge incoming sets and compare the true batch target against the union of single-edge targets. The agent-native evidence is E16: executed LangGraph workflows whose runtime events are normalized into signed delegation decisions and replayed with target proofs. E17/E18 are small A2A-, AutoGen-, and CrewAI-shaped portability artifacts, and E19 is a pinned CrewAI executable workflow with a deterministic local LLM. The mined LangGraph corpus and non-agent DAGs are used only as topology proxies, not as production revocation traces.

VI. RESULTS

A. RQ1: Witness Failures

On the shared-subDAG witness $\text{root} \rightarrow A, B; A, B \rightarrow C; C \rightarrow D$, the semantic target for revoking $A \rightarrow C$ is empty because C and D remain reachable through $B \rightarrow C$. Tree cascade revokes $\{C, D\}$, producing $\text{OverRev} = 2$.

On the cross-deployer witness $\text{root}(d_1) \rightarrow A(d_1) \rightarrow C(d_1) \rightarrow D(d_2)$, the semantic target for revoking $A \rightarrow C$ is $\{C, D\}$. A deployer- d_1 -scoped cascade revokes only $\{C\}$, producing $\text{UnderRev} = 1$.

These witnesses are small by design. Their role is not to establish frequency; it is to show that the two failure modes are deterministic consequences of topology and visibility assumptions.

B. RQ2: Scale-Graph Stress Tests

Generated DAGs are stress tests, not prevalence estimates. In Barabasi-Albert [18] DAGs with $n \in \{100, 500, 2000\}$ and attachment parameter $m \in \{2, 4\}$, tree cascade over-revokes on every sampled revocation event; mean collateral ranges from 5.9 nodes ($n = 100, m = 2$) to 79.7 nodes ($n = 2000, m = 4$). Erdos-Renyi checks with matched average degree show the same qualitative behavior. To rule out cherry-picked edges, we also sample 200 random multi-parent DAGs under five edge-selection policies (uniform, into-multi-parent, root-adjacent, deepest-parent, largest-cascade); tree cascade over-revokes on 63.5–100% of selected edges.

TABLE III

BATCH REVOCATION CANNOT BE IMPLEMENTED AS THE UNION OF EXACT SINGLE-EDGE TARGETS. RANDOM DAGS SAMPLE 2–4-EDGE INCOMING SETS; THE RBAC-INSPIRED ROWS ARE CONCRETE MULTI-PARENT BINDINGS FROM THE CASE STUDY.

Setting	Cases	Failure%	Mean missed	Max missed
Random multi-parent DAGs	39,991	21.25	0.24	5
RBAC: backend-sa parents	1	100.0	3.00	3
RBAC: ci-bot parents	1	100.0	3.00	3
RBAC: ci-step-deploy parents	1	100.0	1.00	1

C. RQ3: Batch Revocation

The RBAC-inspired case study contains a backend service account with two parent bindings. Revoking either parent edge alone leaves an alternate path, so both single-edge targets are empty. Revoking both parent edges simultaneously disconnects the backend node and its workers, producing a three-node batch target. A system that unions independently computed single-edge targets would revoke nothing.

We repeated this test systematically. In 200 random multi-parent DAGs with 80 nodes and shared-parent ratio 0.45, we sampled up to 200 candidate revocation sets per graph from 2–4 incoming edges of multi-parent nodes. Across 39,991 sampled edge sets, the union of exact single-edge targets under-revoked the true batch target in a mean 21.25% of cases per graph (bootstrap 95% CI [20.86, 21.65]; mean missed nodes 0.24; maximum 5). The gap is not universal: many sampled batches are already captured by one constituent edge target. The important point is that a nonzero, reproducible under-revocation gap remains even when every single-edge target is individually exact.

We treat batch non-compositionality as a correctness corner case rather than a prevalence claim. In the mined LangGraph corpus, 25/126 alternate-parent pairs exhibit it, but its operational significance is clearer than its frequency: it is exactly the case that arises when alternate parents are revoked together during scope teardown, parent-key compromise, or policy rollback. The target language must support batch edge sets natively and compute $\text{reach}(G) \setminus \text{reach}(G \setminus R)$ directly; otherwise, even a correct single-edge implementation can under-revoke during incident response.

D. RQ4: Agent-Framework Trace Replay

The witnesses and synthetic tests could be dismissed as generator artifacts, so our primary external-validity test uses executed agent-framework workflows. E16 runs seven fixed LangGraph workflow families plus an exhaustive 3×6 customer-support matrix, repeating each workflow/input cell 20 times. Each execution is a compiled `StateGraph`; we record LangGraph runtime-stream events and application-level caller-to-callee invocations, then normalize them into parent-domain-signed delegation decisions. Construct validity is deliberately narrow: a raw runtime event becomes a delegation decision only when it records a caller, callee, permission tuple (tenant, resource, action, constraints), and known deployer labels.

Generic framework stream events are not counted as authority delegations.

This yields 500 framework traces, 10,500 raw runtime events, and 2,000/2,000 valid signed delegation decisions across 20 deployer domains. Topology claims use the 25 workflow/input cells as clusters, not the deterministic repeats: 13/25 cells contain a runtime-observed multi-parent agent and 8/25 contain cross-deployer sharing. Cell-level tree-cascade over-revocation affects a mean 46.9% of edges (bootstrap 95% CI [29.3, 64.8]); deployer-scoped under-revocation affects 61.6% ([45.6, 76.7]).

Signed replay accepts 500/500 target proofs and 500/500 authorized revocations, including 160/160 cross-deployer cases, and rejects 500/500 unauthorized signers and 260/260 omission attacks. For 320 selected single-edge withdrawals with an alternate runtime parent, the edge target preserves the shared agent in all 320 cases while tree cascade revokes it in all 320. Repeats test replay stability, not production prevalence.

We also replay E16 against target-language baselines. Over 1,960 single-edge withdrawals, edge targets and complete-graph credential revalidation are exact in 1,960/1,960 cases, but the latter requires the verifier to hold the complete graph. Holder/node revocation preserves 0/320 alternate-parent agents; APS-style tree cascade also preserves 0/320 and is exact for only 35.7% of edge cases. Deployer-scoped cascade is exact for 49.0% and under-revokes all 880 cases whose semantic target crosses a deployer boundary.

To check that the normalized signed-event schema is not tied to LangGraph, E17 adds a small A2A-shaped task-handoff artifact and E18 adds AutoGen- and CrewAI-shaped task-handoff artifacts. E19 then runs a pinned CrewAI 1.15.1 workflow with a deterministic local BaseLLM: `Crew.kickoff()` completes, 8/8 task callbacks and 8 LLM calls are observed, and 7/7 task-context handoffs normalize into valid signed delegation events. Together E17–E19 emit five portability traces and 25/25 valid signed delegation events through the same schema. Edge targets preserve all 4/4 alternate-parent cases that tree cascade revokes, and deployer-scoped cascade misses all 11/11 cross-domain target cases. We use these artifacts as portability evidence, not as prevalence studies.

We also mine 28 third-party LangGraph applications using recorded GitHub queries, repository commits, and an exclusion policy that removes tests, examples, tutorials, notebooks, and generated output. Static extraction yields 47 graphs (382 nodes, 384 edges): 31/47 contain a multi-parent node; tree cascade deviates on 70.1% of edges; and 25/126 alternate-parent pairs are batch non-compositional. Because static extraction misses dynamic targets, we use the corpus as lower-bound structural evidence, not as a production trace.

VII. PROTOTYPE AND OVERHEAD

The contributions so far are a semantic contract and a measurement of how approximations deviate from it. A reviewer may reasonably ask whether the contract is *enforceable* without an online central authority. We answer with a working prototype (Python, `cryptography` Ed25519) comprising three components. The anonymous artifact wraps the same

TABLE IV
AGENT-NATIVE, BASELINE, PORTABILITY, AND TOPOLOGY EVIDENCE. E16 REPLAY IS PRIMARY; E17–E19 CHECK SCHEMA PORTABILITY; THE MINED CORPUS IS STATIC LOWER-BOUND EVIDENCE.

Source	Units	MP/AP	X-dep unit	Main outcome
E16 design	25 cells, 500 traces	13 MP	8 cells	repeats test stability
E16 signed replay	2,000 decisions	320 AP	160 cases	500/500 proofs verified
E16 mutations	760 attacks	—	—	760/760 rejected
E16 baselines	1,960 edges	320 AP	880 targets	tree 0/320 AP; scoped 49.0% exact
E17 A2A artifact	2 traces, 8 decisions	1 AP	6 targets	same schema validates
E18 shaped artifacts	2 traces, 10 decisions	2 AP	3 targets	same schema validates
E19 CrewAI executable	1 trace, 7 decisions	1 AP	2 targets	kickoff + schema validates
OSS LangGraph corpus	47 graphs	31 MP	—	70.1% tree deviation

components in a small Python API and command-line verifier that load normalized agent events, issue an edge revocation, export a portable proof bundle, and verify it without the original graph.

a) *Signed delegation graph.*: Every edge is signed by the parent node’s deployer key. A relying party pins deployer public keys out of band and authenticates each edge locally (Section III-D). Edge establishment is bilateral and is part of the protocol, not a free property: a delegation edge becomes valid only after the parent deployer signs the edge descriptor *and* the child deployer updates and signs its accepted-in-edge commitment to include the new edge. This second signature is what later makes adjacency completeness checkable; a node that never accepted an edge will not have committed to it.

b) *Rollback-evident revocation log.*: Revocation events are edge-targeted, signed envelopes appended to a log that publishes a signed checkpoint containing the entry count, a history hash, and the previous checkpoint hash. A relying party that has observed checkpoint *c* rejects a later log with a smaller count, a substituted same-count history, or a broken checkpoint chain. In the prototype, an honest extended log is accepted and rollback/substitution is detected.

c) *Proof-carrying target set with completeness.*: Rather than ship the whole graph, an issuer can ship the claimed target *S* plus *survivor witnesses*: for each reachable node not in *S*, one revocation-free root path of signed edges (proving it survives). Disclosing only the in-edges of target nodes is *not* sound on its own: a malicious issuer could omit an unrevoked in-edge to make a live node look disconnected. We close this with *authenticated adjacency completeness*. Because a node accepts each delegation offered to it, the node’s own deployer maintains and signs a commitment to that node’s *complete* accepted in-edge set. The proof includes this signed commitment for every target node, and the verifier accepts *S* iff (i) every survivor witness authenticates and is revocation-free, (ii) no target node appears as a survivor, (iii) the disclosed cut in-edges of each target node *exactly match* its signed commitment (no omission), and (iv) every unrevoked in-edge of a target node originates from another target node. Under the same trust assumption as edge authenticity—a deployer does not sign evidence that contradicts the delegations it accepted—omission is detectable. In an adversarial test that forges a target by revoking one in-edge and omitting a surviving alternate parent, the verifier rejects all 40/40 attempts.

TABLE V
PROTOTYPE OVERHEAD (MEDIAN OVER 30 GENERATED GRAPHS).

Nodes	Target ms	Proof verify ms	Full view ms	Proof/full bytes
200	0.17	45.9	61.0	64.7%
500	0.43	113.4	155.3	66.0%
1,000	0.91	229.7	310.8	67.0%
2,000	2.15	463.6	644.3	69.0%

d) Overhead.: On generated multi-parent DAGs (45% shared-parent ratio; medians over 30 graph seeds), target computation is linear and takes 0.17–2.15 ms up to 2,000 nodes (Table V). The dominant cost is Ed25519 verification, not graph traversal: cold proof verification remains below full-view verification, and the proof bundle is 64.7–69.0% of the full signed bundle, saving roughly one third of graph disclosure.

The attack matrix rejects 1,100/1,100 malformed envelopes, commitments, Merkle positions, log histories, and unauthorized revocation signers. E16 adds 760/760 replay-level rejection tests for unauthorized signers and omitted in-edges.

The prototype does not solve propagation: it assumes the verifier has observed the relevant revocation events. It does demonstrate that, once events are observed, the semantic target is cheap to compute and check from signed evidence alone, and that rollback of the revocation log is detectable.

VIII. DESIGN IMPLICATIONS

Four implications follow. **Use edge identifiers as the target language**: revocation envelopes should identify delegation edges, not only nodes or issuers, which remain useful for key compromise but cannot remove a single delegation relationship. **Carry graph evidence or compute against a graph view**: a verifier can compute $T_{\text{intent}}(e, G)$ only with enough graph context, satisfiable by (i) a full signed graph view—simple and supporting arbitrary batches, but exposing metadata and needing fresh caches; (ii) an authenticated, gossiped graph cache—local once fresh, but requiring the propagation layer to bound staleness and detect suppression; or (iii) a proof-carrying target set shipping inclusion evidence (each target node loses all root paths) and exclusion evidence (each omitted reachable node keeps a surviving path), for which this paper fixes the statement to be proved, $S = \text{reach}(G) \setminus \text{reach}(G \setminus R)$. **Handle batch revocation natively**: concurrent revocations cannot be reduced to the union of single-edge targets, so APIs and logs should accept edge-id sets and compute $\text{reach}(G) \setminus \text{reach}(G \setminus R)$ directly. **Separate target correctness from delivery**: suppression, gossip, transparency anchoring, and Byzantine delivery are necessary but should propagate the edge-level target semantics rather than invent a different target language.

IX. LIMITATIONS

The policy studied here is disjunctive authority. Systems that require all parents, k -of- n parents, or path-scoped authority need different target sets. E16 uses real LangGraph execution machinery, but the workloads are controlled replay cells rather

than production user traffic, and the node callables stand in for full LLM-backed agents. E17/E18 are A2A-, AutoGen-, and CrewAI-shaped portability artifacts. E19 executes real CrewAI objects, but with a deterministic local LLM and a tiny workflow rather than production traffic. The third-party LangGraph corpus is recovered by static extraction and is therefore a lower bound on connectivity; it can miss dynamic routing and runtime handoffs. Capturing instrumented production agent-delegation traces at scale—across heterogeneous frameworks and real traffic—remains open. The prototype assumes events are already observed and does not implement network propagation. A propagation layer should still budget stale authority: an agent executing at rate v can perform work on the order of $v(\text{TTL} + \delta_{\text{prop}})$ before a revocation becomes visible, but this paper fixes the target that such a layer must deliver.

X. RELATED WORK

a) Delegation graphs and revocation semantics.: Treating delegated authority as graph-structured evidence is well established in trust management—decentralized trust management [19], SPKI/SDSI authorization and chain discovery [20], [21], and role-based trust management [22]. Revocation has likewise been studied as a semantic policy question: Hagström et al. classify cascading versus non-cascading and immediate versus eventual revocation [23], and DW-RBAC formalizes delegation/revocation with cascading under a centralized policy [14]. We follow this tradition but change the setting: a federated, multi-parent agent DAG enforced verifier-independently from signed edges and revocation events rather than a central engine, with an explicit edge-level target.

b) Agent identity and delegated authority.: Recent agent-identity work makes delegation a first-class AI-agent concern. South et al. propose authenticated, authorized, and auditable authority for AI agents on top of OAuth/OIDC infrastructure [6]; the OpenID Foundation whitepaper identifies delegated authority, agent-native identity, and agent lifecycle management as central identity-management problems for agentic AI [7]. AIP adds verifiable delegation across MCP and A2A using invocation-bound capability tokens [8]. These systems authenticate who delegated what authority, but they do not define the exact authority loss caused by revoking one edge in a multi-parent agent workflow.

c) Agent protocols and access-control enforcement.: AutoGen, CrewAI, and LangGraph provide orchestration abstractions for composing multi-agent workflows [1]–[3]. MCP specifies OAuth-style authorization for protected tool servers [5], and A2A standardizes cross-agent discovery and communication [4]. Protocol threat modeling for MCP, A2A, Agora, and ANP shows that agent protocols create new trust and lifecycle risk surfaces [24]. Complementary work on agent access control argues for dynamic, context-aware information-flow governance [25]; SEAgent enforces MAC/ABAC policies to prevent privilege escalation in LLM-based agent systems [26]; and compositional governance frameworks formalize delegation and scope as agentic authorization primitives [27]. These works constrain or govern agent actions at runtime. Our

work is narrower and complementary: it specifies which agents should lose authority after an edge withdrawal and how a verifier checks that target from signed evidence.

d) Credentials, capabilities, and tree passports.:

CRLs [9], OCSP [10], CRLite [11], Macaroons [12], and SPIFFE/SPIRE [13] either revoke credentials/tokens or bound authority with short lifetimes. APS is closest among agent proposals but uses a single-parent rooted tree whose cascade semantics cannot preserve alternate parents [16]. None defines a multi-parent, cross-domain edge target (Table I).

e) Dominators.: Our target $\text{reach}(G) \setminus \text{reach}(G \setminus \{e\})$ is a root-relative edge-dominance set, computable with standard dominator algorithms [28], [29]; we claim not a new algorithm but the identification of this set as the revocation contract that signed agent delegation must meet.

XI. CONCLUSION

AI-agent revocation is an authority-consistency problem, not only a token lifecycle problem. In a federated multi-parent workflow, revoking one delegation edge need not revoke the child agent, while a local cascade can miss cross-deployer descendants that still depend on the withdrawn relationship. This paper makes the target explicit: under disjunctive authority, edge revocation invalidates exactly $\text{reach}(G) \setminus \text{reach}(G \setminus \{e\})$. Used as the reference target, this contract exposes over-revocation for tree cascades, under-revocation for deployer-scoped cascades, and non-compositional behavior for batch withdrawals. The practical implication for agent protocols is simple: signed delegation evidence should identify edges, revocation APIs should accept edge sets, and relying parties should receive enough authenticated graph evidence to check the authority loss they are asked to enforce.

REFERENCES

- [1] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155; Oral, ICLR 2024 Workshop on LLM Agents, 2023.
- [2] CrewAI, Inc., "CrewAI: Framework for orchestrating role-playing, autonomous AI agents," <https://crewai.com>, 2024.
- [3] LangChain, "LangGraph: Graph API," <https://docs.langchain.com/oss/python/langgraph/graph-api>, 2026, documentation accessed 2026.
- [4] A2A Project, "Agent2Agent (A2A) protocol specification," <https://a2a-protocol.org/latest/specification/>, 2025, open protocol contributed by Google and hosted by the Linux Foundation.
- [5] Model Context Protocol, "Authorization," <https://modelcontextprotocol.io/specification/2025-11-25/basic/authorization>, 2025, specification revision 2025-11-25.
- [6] T. South, S. Marro, T. Hardjono, R. Mahari, C. D. Whitney, D. Greenwood, A. Chan, and A. Pentland, "Authenticated delegation and authorized AI agents," arXiv:2501.09674, 2025.
- [7] T. South, S. Nagabhushanaradhya, A. Dissanayaka, S. Cecchetti, G. Fletcher, V. Lu, A. Pietropaolo, D. H. Saxe, J. Lombardo, A. M. Shivalingaiah, S. Bounev, A. Keisner, A. Kesselman, Z. Proser, G. Fahs, A. Bunyea, B. Moskowitz, A. Tulshibagwale, D. Greenwood, J. Pei, and A. Pentland, "Identity management for agentic AI: The new frontier of authorization, authentication, and security for an AI agent world," OpenID Foundation whitepaper; arXiv:2510.25819, 2025.
- [8] S. Prakash, "AIP: Agent identity protocol for verifiable delegation across MCP and A2A," arXiv:2603.24775, 2026.
- [9] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, and W. Polk, "Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile," IETF, RFC 5280, 2008.
- [10] S. Santesson, M. Myers, R. Ankney, A. Malpani, S. Galperin, and C. Adams, "X.509 Internet Public Key Infrastructure Online Certificate Status Protocol – OCSP," IETF, RFC 6960, 2013.
- [11] J. Larisch, D. Choffnes, D. Levin, B. M. Maggs, A. Mislove, and C. Wilson, "CRLite: A scalable system for pushing all TLS revocations to all browsers," in *Proceedings of the 2017 IEEE Symposium on Security and Privacy (S&P)*, 2017, pp. 539–556.
- [12] A. Birgisson, J. G. Politz, Ú. Erlingsson, A. Taly, M. Vrabie, and M. Lentzner, "Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud," in *Proceedings of NDSS*, 2014.
- [13] C. S. W. Group, "SPIFFE: Secure production identity framework for everyone," Cloud Native Computing Foundation, Tech. Rep., 2018, <https://spiffe.io/>.
- [14] J. Wainer, A. Kumar, and P. Barthelmeß, "DW-RBAC: A formal security model of delegation and revocation in workflow systems," *Information Systems*, vol. 32, no. 3, pp. 365–384, 2007.
- [15] T. Lodderstedt, S. Dronia, and M. Scurtescu, "OAuth 2.0 Token Revocation," IETF, RFC 7009, 2013.
- [16] T. Pidlisnyi, "Agent passport system (APS)," <https://datatracker.ietf.org/doc/draft-pidlisnyi-aps/>, Feb. 2026, IETF Internet-Draft, work in progress; project repository: <https://github.com/aeoess/agent-passport-system>.
- [17] The Kubernetes Authors, "Using RBAC authorization," Kubernetes Documentation, 2024, <https://kubernetes.io/docs/reference/access-authn-authz/rbac/>.
- [18] A.-L. Barabási and R. Albert, "Emergence of scaling in random networks," *Science*, vol. 286, no. 5439, pp. 509–512, 1999.
- [19] M. Blaze, J. Feigenbaum, and J. Lacy, "Decentralized trust management," in *Proceedings of the 1996 IEEE Symposium on Security and Privacy*, 1996, pp. 164–173.
- [20] D. Clarke, C. Ellison, M. Fredette, A. Morcos, and R. L. Rivest, "Certificate chain discovery in SPKI/SDSI," *Journal of Computer Security*, vol. 9, no. 4, pp. 285–322, 2001.
- [21] J. Y. Halpern and R. van der Meyden, "A logical reconstruction of SPKI," arXiv:cs/0208028, 2002.
- [22] N. Li, J. C. Mitchell, and W. H. Winsborough, "Design of a role-based trust-management framework," in *Proceedings of the 2002 IEEE Symposium on Security and Privacy*, 2002, pp. 114–130.
- [23] Å. Hagström, S. Jajodia, F. Parisi-Presicce, and D. Wijesekera, "Revocations — a classification," in *Proceedings of the 14th IEEE Computer Security Foundations Workshop (CSFW)*, 2001, pp. 44–58.
- [24] Z. Anbiaee, M. Rabbani, M. Mirani, G. Piya, I. Opushnyev, A. Ghorbani, and S. Dadkhah, "Security threat modeling for emerging AI-agent protocols: A comparative analysis of MCP, A2A, Agora, and ANP," arXiv:2602.11327, 2026.
- [25] X. Li, D. Huang, J. Li, H. Cai, Z. Zhou, W. Dong, X. Wang, and Y. Liu, "A vision for access control in LLM-based agent systems," arXiv:2510.11108, 2025.
- [26] Z. Ji, D. Wu, W. Jiang, P. Ma, Z. Li, Y. Gao, S. Wang, and Y. Li, "Taming various privilege escalation in LLM-based agent systems: A mandatory access control framework," arXiv:2601.11893, 2026.
- [27] A. Ibrahim and Y. Li, "Overlaying governance: A compositional authorization framework for delegation and scope in agentic AI," arXiv:2606.03518, 2026.
- [28] T. Lengauer and R. E. Tarjan, "A fast algorithm for finding dominators in a flowgraph," *ACM Transactions on Programming Languages and Systems*, vol. 1, no. 1, pp. 121–141, 1979.
- [29] L. Georgiadis, G. F. Italiano, L. Laura, and F. Santaroni, "An experimental study of dynamic dominators," arXiv:1604.02711, 2016.